Handwritten Notes - Problems and Codes Only

Created from all uploaded Python question-paper PDFs

Paper Number Mapping

P1 - PROBLEM-SOLVING-USING-PYTHON-AUG-2022.pdf
P2 - PROBLEM-SOLVING-USING-PYTHON-JAN-2023.pdf
P3 - PYTHON PROGRAMMING SEPT 2021.pdf
P4 - PYTHON-PROGRAMMING-AUG-2022 (1).pdf - contains SET 1 to SET 4
P5 - PYTHON-PROGRAMMING-JAN-2023.pdf
P6 - PYTHON-PROGRAMMING-MARCH-2022.pdf

Every code item below shows Paper number, Unit number and Question number. Repeated coding questions are combined where the same answer fits.

Unit-I - Coding Problems

Source: P1 U1 Q1(a)

Problem: Display Fibonacci sequence up to nth term given by user.

```
n = int(input("Enter number of terms: "))
a, b = 0, 1
print("Fibonacci sequence:")
for i in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

Source: P1 U1 Q2(a)

Problem: Find the sum of all odd and even numbers up to a number specified by user.

```
n = int(input("Enter n: "))
even_sum = 0
odd_sum = 0
for i in range(1, n + 1):
    if i % 2 == 0:
        even_sum += i
    else:
        odd_sum += i
print("Even sum =", even_sum)
print("Odd sum =", odd_sum)
```

Source: P2 U1 Q1(c), P4-S1 U1 Q2(b)

Problem: Demonstrate logical, membership and identity operators.

```
a, b = 10, 20
nums = [10, 30, 50]
x = nums
y = [10, 30, 50]

print(a < b and b > 15)       # logical and
print(a > b or b == 20)       # logical or
print(not(a == b))            # logical not
print(10 in nums)             # membership
print(40 not in nums)
print(x is nums)              # identity True
print(x is y)                 # identity False (different objects)
print(x is not y)
```

Source: P2 U1 Q2(b), P4-S2 U1 Q1(b)

Problem: Calculate final amount after 10% discount when purchase amount is more than 1000.

```
amount = float(input("Enter purchase amount: "))
if amount > 1000:
    discount = amount * 0.10
else:
    discount = 0
payable = amount - discount
print("Discount =", discount)
print("Amount to pay =", payable)
```

Source: P3 U1 Q2(a), P5 U1 Q1(b)

Problem: Demonstrate explicit type conversion using int(), float() and str().

```
s = "25"
n = int(s)
f = float(s)
msg = str(n)
print("String:", s, type(s))
print("Integer:", n, type(n))
print("Float:", f, type(f))
print("Back to string:", msg, type(msg))
```

Source: P3 U1 Q2(b), P4-S3 U2 Q4(a)

Problem: Demonstrate break and continue in loop control statements.

```
print("Using continue: skip 5")
for i in range(1, 11):
    if i == 5:
        continue
    print(i, end=" ")

print("\nUsing break: stop at 5")
for i in range(1, 11):
    if i == 5:
        break
    print(i, end=" ")
```

Source: P4-S1 U1 Q2(a)

Problem: Access characters in a Python string.

```
s = "Python"
print("First char:", s[0])
print("Last char:", s[-1])
print("Middle chars:", s[1:4])
for ch in s:
    print(ch, end=" ")
```

Source: P4-S3 U1 Q2(a)

Problem: Demonstrate format() method specifiers.

```
name = "Ravi"
marks = 88.567
print("Name: {}".format(name))
print("Marks: {:.2f}".format(marks))
print("Right align: {:>10}".format(name))
print("Left align: {:<10}".format(name))
print("Binary of 10: {:b}".format(10))
```

Source: P4-S4 U1 Q1(b)

Problem: Find numbers divisible by 5 and multiples of 3 between 1 and 1000.

```
for n in range(1, 1001):
    if n % 5 == 0 and n % 3 == 0:
        print(n, end=" ")
```

Source: P5 U1 Q2(b)

Problem: Multiply two matrices using nested loops.

```
A = [[1, 2, 3], [4, 5, 6]]
B = [[7, 8], [9, 10], [11, 12]]
rows_A, cols_A = len(A), len(A[0])
cols_B = len(B[0])
result = [[0 for j in range(cols_B)] for i in range(rows_A)]

for i in range(rows_A):
    for j in range(cols_B):
        for k in range(cols_A):
            result[i][j] += A[i][k] * B[k][j]

for row in result:
    print(row)
```

Source: P6 U1 Q1(b)

Problem: Print first N numbers divisible by 5 using while loop.

```
n = int(input("How many numbers? "))
count = 0
num = 5
while count < n:
    print(num, end=" ")
    num += 5
    count += 1
```

Source: P6 U1 Q2(b)

Problem: Find best of two test average marks out of three test marks.

```
m1 = float(input("Test 1 marks: "))
m2 = float(input("Test 2 marks: "))
m3 = float(input("Test 3 marks: "))
lowest = min(m1, m2, m3)
best_two_avg = (m1 + m2 + m3 - lowest) / 2
print("Best of two average =", best_two_avg)
```

Unit-II - Coding Problems

Source: P2 U2 Q3(a)

Problem: Print all prime numbers from 1 to 100 except even primes.

```
for n in range(2, 101):
    if n == 2:        # 2 is even prime, so skip it
        continue
    is_prime = True
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            is_prime = False
            break
    if is_prime:
        print(n, end=" ")
```

Source: P3 U2 Q3(a)

Problem: Compute sum of even numbers within given natural number using continue.

```
n = int(input("Enter n: "))
s = 0
for i in range(1, n + 1):
    if i % 2 != 0:
        continue
    s += i
print("Sum of even numbers =", s)
```

Source: P3 U2 Q4(b)

Problem: Check whether a number is Armstrong number or not.

```
n = int(input("Enter number: "))
temp = n
power = len(str(n))
sum_digits = 0
while temp > 0:
    digit = temp % 10
    sum_digits += digit ** power
    temp //= 10

if sum_digits == n:
    print(n, "is Armstrong")
else:
    print(n, "is not Armstrong")
```

Source: P4-S1 U2 Q4(b)

Problem: Display Fibonacci series between 0 and 100.

```
a, b = 0, 1
while a <= 100:
    print(a, end=" ")
    a, b = b, a + b
```

Source: P4-S2 U2 Q4(a)

Problem: Print numbers from 0 to 100 except multiples of 4 and 6.

```
for n in range(0, 101):
    if n % 4 == 0 or n % 6 == 0:
        continue
    print(n, end=" ")
```

Source: P4-S3 U2 Q3(a)

Problem: Print star pattern using nested for loop.

```
# Pattern: *, **, ***, ****, *****, ****, ***, **, *
for i in range(1, 6):
    print("*" * i)
for i in range(4, 0, -1):
    print("*" * i)
```

Source: P4-S4 U2 Q3(b)

Problem: Get substring between two characters in the input string.

```
s = input("Enter string: ")
start = input("Start character: ")
end = input("End character: ")

p1 = s.find(start)
p2 = s.find(end, p1 + 1)
if p1 != -1 and p2 != -1:
    print("Substring =", s[p1 + 1:p2])
else:
    print("Characters not found")
```

Source: P4-S4 U2 Q4(a)

Problem: Compute natural logarithm of 2 using series 1 - 1/2 + 1/3 - 1/4 + ... up to n terms.

```
n = int(input("Enter number of terms: "))
log2 = 0
for i in range(1, n + 1):
    if i % 2 == 1:
        log2 += 1 / i
    else:
        log2 -= 1 / i
print("Approximate ln(2) =", log2)
```

Source: P5 U2 Q4(a)

Problem: Reverse a number and find sum of digits in the reversed number.

```
n = int(input("Enter number: "))
rev = 0
sum_digits = 0
while n > 0:
    digit = n % 10
    rev = rev * 10 + digit
    sum_digits += digit
    n //= 10
print("Reversed number =", rev)
print("Sum of digits =", sum_digits)
```

Source: P6 U2 Q3(b)

Problem: Traverse through a string using a loop.

```
s = input("Enter a string: ")
for ch in s:
    print(ch)
```

Source: P6 U2 Q4(a)

Problem: Print multiplication table of a given number.

```
n = int(input("Enter number: "))
for i in range(1, 11):
    print(n, "x", i, "=", n * i)
```

Source: P1 U2 Q4(b)

Problem: Search whether a given substring is present in a string.

```
text = input("Enter main string: ")
sub = input("Enter substring: ")
if sub in text:
    print("Substring found at index", text.find(sub))
else:
    print("Substring not found")
```

Unit-III - Coding Problems

Source: P2 U3 Q5(a)

Problem: Remove 0th, 4th and 5th elements from a list.

```
items = ['Red', 'Green', 'White', 'Black', 'Pink', 'Yellow']
# remove from highest index to lowest to avoid index shift
for index in [5, 4, 0]:
    items.pop(index)
print(items)      # ['Green', 'White', 'Black']
```

Source: P3 U3 Q5(b)

Problem: Read one subject mark and print pass/fail using function with argument and single return value.

```
def result(mark):
    if mark >= 35:
        return "Pass"
    else:
        return "Fail"

m = int(input("Enter marks: "))
print(result(m))
```

Source: P3 U3 Q6(b)

Problem: Read a word and print number of letters, vowels and percentage of vowels using dictionary.

```
word = input("Enter word: ")
vowels = "aeiouAEIOU"
info = {}
info["letters"] = len(word)
info["vowels"] = sum(1 for ch in word if ch in vowels)
info["vowel_percent"] = (info["vowels"] / info["letters"]) * 100
print(info)
```

Source: P4-S1 U3 Q5(a)

Problem: Demonstrate common list built-in methods.

```
nums = [10, 20, 30]
nums.append(40)
nums.insert(1, 15)
nums.extend([50, 60])
print(nums)
nums.remove(20)
print("Count of 30:", nums.count(30))
print("Index of 40:", nums.index(40))
nums.sort()
print("Sorted:", nums)
nums.reverse()
print("Reverse:", nums)
```

Source: P4-S1 U3 Q5(b)

Problem: Function to print even numbers from a given list.

```
def print_even(nums):
    for n in nums:
        if n % 2 == 0:
            print(n, end=" ")

print_even([1, 2, 3, 4, 5, 6, 8])
```

Source: P4-S1 U3 Q6(a)

Problem: Access elements of a nested dictionary.

```
students = {
    "101": {"name": "Asha", "marks": 88},
    "102": {"name": "Ravi", "marks": 91}
}
print(students["101"]["name"])
print(students["102"]["marks"])

for roll, data in students.items():
    print(roll, data["name"], data["marks"])
```

Source: P4-S2 U3 Q5(b), P1 U3 Q6(a), P2 U3 Q6(a)

Problem: Demonstrate lambda / anonymous function.

```
square = lambda x: x * x
add = lambda a, b: a + b
print(square(5))
print(add(10, 20))

nums = [1, 2, 3, 4]
squares = list(map(lambda x: x * x, nums))
print(squares)
```

Source: P4-S3 U3 Q5(b)

Problem: Check whether a number is perfect or not using function.

```
def is_perfect(n):
    s = 0
    for i in range(1, n):
        if n % i == 0:
            s += i
    return s == n

num = int(input("Enter number: "))
if is_perfect(num):
    print("Perfect number")
else:
    print("Not perfect")
```

Source: P5 U3 Q5(a)

Problem: Use function to check whether a string is palindrome.

```
def is_palindrome(s):
    s = s.lower().replace(" ", "")
    return s == s[::-1]

text = input("Enter string: ")
if is_palindrome(text):
    print("Palindrome")
else:
    print("Not palindrome")
```

Source: P5 U3 Q5(b)

Problem: Replace numbers divisible by 3, 5, and both in list with ppp/qqq/pppqqq.

```
num = [45, 22, 14, 65, 97, 72]
for i in range(len(num)):
    if num[i] % 3 == 0 and num[i] % 5 == 0:
        num[i] = "pppqqq"
    elif num[i] % 3 == 0:
        num[i] = "ppp"
    elif num[i] % 5 == 0:
        num[i] = "qqq"
print(num)
```

Source: P5 U3 Q6(a)

Problem: Create a list with computer languages and display using while loop.

```
languages = ["Python", "C", "Java", "C++", "JavaScript"]
i = 0
while i < len(languages):
    print(languages[i])
    i += 1
```

Source: P6 U3 Q6(a)

Problem: Pass a list into a function.

```
def display_items(items):
    for item in items:
        print(item)

subjects = ["Python", "Maths", "Physics"]
display_items(subjects)
```

Source: P6 U3 Q6(b)

Problem: Function that takes two lists and returns True if they have at least one common member.

```
def has_common(list1, list2):
    for x in list1:
        if x in list2:
            return True
    return False

print(has_common([1, 2, 3], [5, 6, 2]))
print(has_common([1, 2], [3, 4]))
```

Source: P4-S1 U3 Q6(b), P5 U3 Q6(b)

Problem: Import specific attributes and rename a module.

```
# Import specific item from module
from math import sqrt
print(sqrt(49))

# Rename module using alias
import math as m
print(m.pi)
print(m.factorial(5))
```

Source: P1 U3 Q5(b), P6 U3 Q5(a)

Problem: Create and use a simple package.

```
# Folder structure:
# mypack/
#     __init__.py
#     calc.py

# calc.py
# def add(a, b):
#     return a + b

# main.py
from mypack.calc import add
print(add(10, 20))
```

Unit-IV - Coding Problems

Source: P3 U4 Q7(a)

Problem: Create Employee class with name, department and salary; read and print details.

```
class Employee:
    def read(self):
        self.name = input("Enter name: ")
        self.department = input("Enter department: ")
        self.salary = float(input("Enter salary: "))

    def display(self):
        print("Name:", self.name)
        print("Department:", self.department)
        print("Salary:", self.salary)

emp = Employee()
emp.read()
emp.display()
```

Source: P1 U4 Q7(b), P5 U4 Q7(b)

Problem: Define a class, create objects and access members.

```
class Student:
    def __init__(self, name, roll):
        self.name = name
        self.roll = roll

    def display(self):
        print("Name:", self.name)
        print("Roll:", self.roll)

s1 = Student("Ravi", 101)
print(s1.name)       # accessing data member
s1.display()         # calling method
```

Source: P4-S4 U4 Q8(a)

Problem: Create student class with name, branch and grade; read and display details.

```
class Student:
    def read(self):
        self.name = input("Name: ")
        self.branch = input("Branch: ")
        self.grade = input("Grade: ")

    def display(self):
        print("Name:", self.name)
        print("Branch:", self.branch)
        print("Grade:", self.grade)

s = Student()
s.read()
s.display()
```

Source: P4-S1 U4 Q7(b)

Problem: Class inheritance with issubclass() and isinstance().

```
class Animal:
    pass

class Dog(Animal):
    pass

d = Dog()
print(issubclass(Dog, Animal))   # True
print(isinstance(d, Dog))        # True
print(isinstance(d, Animal))     # True
```

Source: P6 U4 Q8(a)

Problem: Demonstrate inheritance and super() function.

```
class Person:
    def __init__(self, name):
        self.name = name

    def show(self):
        print("Name:", self.name)

class Student(Person):
    def __init__(self, name, roll):
        super().__init__(name)
        self.roll = roll

    def show(self):
        super().show()
        print("Roll:", self.roll)

s = Student("Ravi", 101)
s.show()
```

Source: P4-S2 U4 Q8(b), P3 U4 Q7(b)

Problem: Illustrate method overriding and simple overloading using default arguments.

```
class Animal:
    def sound(self):
        print("Some sound")

class Dog(Animal):
    def sound(self):       # overriding parent method
        print("Bark")

class Calculator:
    def add(self, a, b=0, c=0):   # overloading style
        return a + b + c

Dog().sound()
obj = Calculator()
print(obj.add(5))
print(obj.add(5, 10))
print(obj.add(5, 10, 15))
```

Source: P4-S1 U4 Q8(b), P5 U4 Q8(a), P6 U4 Q7(b)

Problem: Overload + and - operators using __add__ and __sub__.

```
class Number:
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return Number(self.value + other.value)

    def __sub__(self, other):
        return Number(self.value - other.value)

    def __str__(self):
        return str(self.value)

a = Number(20)
b = Number(5)
print(a + b)    # 25
print(a - b)    # 15
```

Source: P6 U4 Q8(b)

Problem: Create a destructor in Python.

```
class Demo:
    def __init__(self):
        print("Object created")

    def __del__(self):
        print("Destructor called")

obj = Demo()
del obj
```

Source: P3 U4 Q8(a)

Problem: Read a text file and capitalize each character of file.

```
filename = input("Enter file name: ")
with open(filename, "r") as f:
    data = f.read()

with open(filename, "w") as f:
    f.write(data.upper())

print("File content converted to uppercase")
```

Source: P1 U4 Q8(b), P2 U2 Q4(b), P4-S3 U2 Q4(b)

Problem: Read/write a text file or store file lines into a list.

```
# Write data to a file
with open("sample.txt", "w") as f:
    f.write("Python is easy\n")
    f.write("File handling example\n")

# Read file line by line into a list
lines = []
with open("sample.txt", "r") as f:
    for line in f:
        lines.append(line.strip())

print(lines)
```

Source: P4-S1 U4 Q8(a), P4-S3 U4 Q7(b)

Problem: Manipulate file pointer using seek() and tell().

```
with open("sample.txt", "w+") as f:
    f.write("Hello Python")
    print("Current position:", f.tell())
    f.seek(0)
    print("After seek:", f.tell())
    print("Data:", f.read())
```

Source: P4-S4 U2 Q4(b)

Problem: Search string "laptop" in sales.txt and print line with line number.

```
word = "laptop"
with open("sales.txt", "r") as f:
    for line_no, line in enumerate(f, start=1):
        if word.lower() in line.lower():
            print(line_no, ":", line.strip())
```

Source: P5 U4 Q7(a)

Problem: Count and print occurrence of each word in a file using dictionary.

```
filename = input("Enter file name: ")
counts = {}
with open(filename, "r") as f:
    for line in f:
        for word in line.split():
            word = word.lower().strip(".,!?;:")
            counts[word] = counts.get(word, 0) + 1
print(counts)
```

Source: P6 U4 Q7(a)

Problem: Find the longest word in a file. Get file name from user.

```
filename = input("Enter file name: ")
longest = ""
with open(filename, "r") as f:
    for line in f:
        for word in line.split():
            word = word.strip(".,!?;:")
            if len(word) > len(longest):
                longest = word
print("Longest word =", longest)
```

Source: P4-S3 U3 Q6(b)

Problem: Extract characters from text files and put them into a list.

```
files = ["a.txt", "b.txt", "c.txt"]
chars = []
for filename in files:
    with open(filename, "r") as f:
        data = f.read()
        for ch in data:
            if ch != "\n":
                chars.append(ch)
print(chars)
```

Source: P2 U4 Q8(a), P4-S3 U4 Q8(b)

Problem: Read config file values in Python.

```
# config.ini:
# [Database]
# host = localhost
# port = 3306

import configparser
config = configparser.ConfigParser()
config.read("config.ini")
host = config["Database"]["host"]
port = config["Database"]["port"]
print(host, port)
```

Unit-V - Coding Problems

Source: P1 U5 Q9(a), P2 U5 Q9(b), P4-S3 U5 Q9(b)

Problem: Single try block with multiple except blocks.

```
try:
    a = int(input("Enter a: "))
    b = int(input("Enter b: "))
    result = a / b
    print("Result =", result)
except ValueError:
    print("Please enter integers only")
except ZeroDivisionError:
    print("Cannot divide by zero")
except Exception as e:
    print("Other error:", e)
```

Source: P1 U5 Q9(b)

Problem: Use assert statement to generate AssertionError.

```
try:
    age = int(input("Enter age: "))
    assert age >= 18, "Age must be at least 18"
    print("Eligible")
except AssertionError as e:
    print("Assertion Error:", e)
```

Source: P1 U5 Q10(a), P4-S2 U5 Q9(a)

Problem: Handle exception using try-except / division by zero.

```
try:
    a = int(input("Enter a: "))
    b = int(input("Enter b: "))
    print("Division =", a / b)
except ZeroDivisionError:
    print("Error: Division by zero is not allowed")
```

Source: P3 U5 Q9(a), P4-S2 U5 Q9(b), P5 U5 Q9(a)

Problem: Difference between else block and finally block with example.

```
try:
    n = int(input("Enter number: "))
    print("Square =", n * n)
except ValueError:
    print("Invalid number")
else:
    print("No exception occurred")
finally:
    print("Finally block always executes")
```

Source: P4-S1 U5 Q9, P6 U5 Q10(a)

Problem: Create, raise and handle user-defined exception.

```
class AgeError(Exception):
    pass

try:
    age = int(input("Enter age: "))
    if age < 0 or age > 120:
        raise AgeError("Age must be between 0 and 120")
    print("Valid age")
except AgeError as e:
    print("Error:", e)
```

Source: P2 U5 Q10(a), P4-S2 U5 Q10(a)

Problem: GUI program to create three single-line text boxes using Tkinter.

```
import tkinter as tk
root = tk.Tk()
root.title("Three Text Boxes")

for i in range(3):
    tk.Label(root, text=f"Value {i+1}:").grid(row=i, column=0, padx=5, pady=5)
    tk.Entry(root).grid(row=i, column=1, padx=5, pady=5)

root.mainloop()
```

Source: P3 U5 Q9(b), P4-S1 U5 Q10(a)

Problem: Create radio buttons for gender selection using Tkinter.

```
import tkinter as tk
root = tk.Tk()
root.title("Gender Selection")

gender = tk.StringVar(value="Male")
for g in ["Male", "Female", "Transgender"]:
    tk.Radiobutton(root, text=g, variable=gender, value=g).pack(anchor="w")

tk.Button(root, text="Show", command=lambda: print(gender.get())).pack()
root.mainloop()
```

Source: P4-S3 U5 Q10(a)

Problem: Create Listbox widget using Tkinter.

```
import tkinter as tk
root = tk.Tk()
root.title("Listbox Example")

lb = tk.Listbox(root, height=5)
for item in ["Python", "Java", "C", "C++", "HTML"]:
    lb.insert(tk.END, item)
lb.pack(padx=10, pady=10)

root.mainloop()
```

Source: P4-S3 U5 Q10(b)

Problem: Create OptionMenu using Tkinter.

```
import tkinter as tk
root = tk.Tk()
root.title("OptionMenu Example")

choice = tk.StringVar(value="Python")
options = ["Python", "Java", "C++", "JavaScript"]
tk.OptionMenu(root, choice, *options).pack(padx=20, pady=20)
tk.Button(root, text="Show", command=lambda: print(choice.get())).pack()

root.mainloop()
```

Source: P5 U5 Q10

Problem: GUI with textbox, OK button and Quit button.

```
import tkinter as tk
root = tk.Tk()
root.title("Text Box GUI")

entry = tk.Entry(root, width=30)
entry.pack(padx=10, pady=10)

def show_text():
    print(entry.get())

tk.Button(root, text="OK", command=show_text).pack(pady=5)
tk.Button(root, text="Quit", command=root.destroy).pack(pady=5)
root.mainloop()
```

Source: P6 U5 Q9(a)

Problem: Basic web browser using Tkinter Text/Canvas area to display page content.

```
import tkinter as tk
from urllib.request import urlopen

root = tk.Tk()
root.title("Mini Browser")
url_entry = tk.Entry(root, width=60)
url_entry.pack(padx=10, pady=5)
canvas = tk.Canvas(root, width=600, height=300, bg="white")
canvas.pack(padx=10, pady=10)

def load_page():
    url = url_entry.get()
    if not url.startswith("http"):
        url = "https://" + url
    try:
        data = urlopen(url).read(1000).decode("utf-8", errors="ignore")
        canvas.delete("all")
        canvas.create_text(10, 10, anchor="nw", text=data[:1000], width=580)
```

```
    except Exception as e:
        canvas.delete("all")
        canvas.create_text(10, 10, anchor="nw", text="Error: " + str(e), width=580)

tk.Button(root, text="Open", command=load_page).pack()
root.mainloop()
```

Source: P4-S4 U5 Q10

Problem: Scrollable Listbox, Menu and Progressbar widgets using Tkinter.

```
import tkinter as tk
from tkinter import ttk
root = tk.Tk()
root.title("Widgets Demo")

# Scrollable Listbox
frame = tk.Frame(root)
frame.pack(padx=10, pady=5)
scroll = tk.Scrollbar(frame)
scroll.pack(side=tk.RIGHT, fill=tk.Y)
listbox = tk.Listbox(frame, yscrollcommand=scroll.set)
for i in range(1, 21):
    listbox.insert(tk.END, "Item " + str(i))
listbox.pack(side=tk.LEFT)
scroll.config(command=listbox.yview)

# Menu
menubar = tk.Menu(root)
file_menu = tk.Menu(menubar, tearoff=0)
file_menu.add_command(label="Exit", command=root.destroy)
menubar.add_cascade(label="File", menu=file_menu)
root.config(menu=menubar)

# Progressbar
bar = ttk.Progressbar(root, length=200, mode="determinate")
bar.pack(pady=10)
bar["value"] = 60
root.mainloop()
```